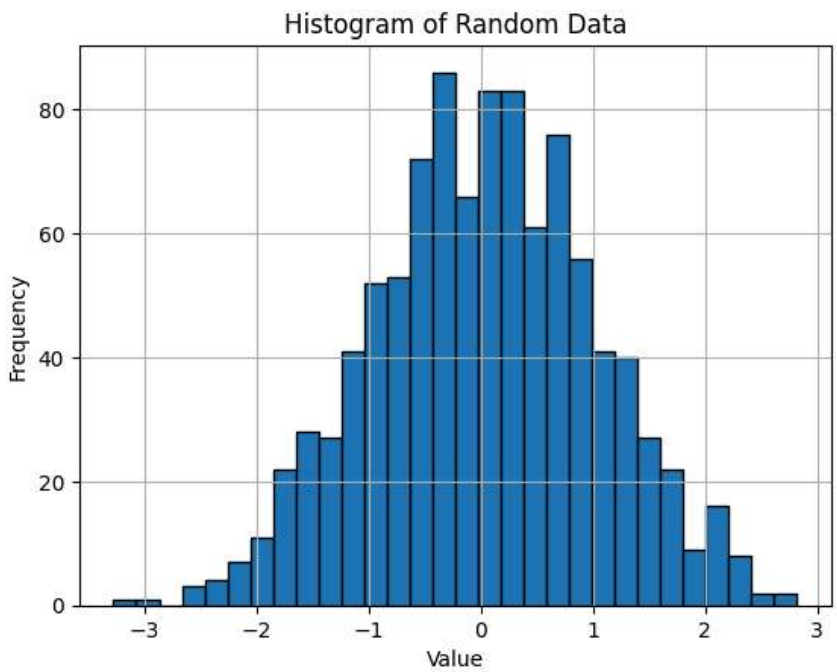
```
import numpy as np
import matplotlib.pyplot as plt

# Generate some random data for the histogram
data = np.random.randn(1000)

# Create the histogram
plt.hist(data, bins=30, edgecolor='black')
plt.title('Histogram of Random Data')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.grid(True)
plt.show()
```




```
import numpy as np
```

```
a = [1,2,3,4,5]
a
```

```
[1, 2, 3, 4, 5]
```

```
arr1 = np.array(a)
arr1
```

```
array([1, 2, 3, 4, 5])
```

```
arr2 = np.array([[1,2,3],[4,5,6]])
arr2
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
arr3 = np.array([[[1,2,3,4],[44,5,6,7]],[[1,3,6,7],[4,5,6,7]]])
arr3
```

```
array([[[ 1,  2,  3,  4],
        [44,  5,  6,  7]],
```

```
       [[ 1,  3,  6,  7],
        [ 4,  5,  6,  7]]])
```

```
np.full ((3,4),2)
```

```
array([[2, 2, 2, 2],
       [2, 2, 2, 2],
       [2, 2, 2, 2]])
```

```
type(arr1)
```

```
numpy.ndarray
```

```
type(arr2)
```

```
numpy.ndarray
```

```
arr1.shape
```

```
(5,)
```

```
arr2.shape
```

```
(2, 3)
```

```
arr3.shape
```

```
(2, 2, 4)
```

```
arr1[4]
```

```
np.int64(5)
```

```
arr1[-1]
```

```
np.int64(5)
```

```
arr1[2:4]
```

```
array([3, 4])
```

```
arr2[1,1]
```

```
np.int64(5)
```

```
arr2[1,2]
```

```
np.int64(6)
```

```
arr2[:,-1]
```

```
array([3, 6])
```

```
arr2
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
arr3[1,:,2:4] ###matrix number,rows,column
```

```
array([[6, 7],
       [6, 7]])
```

```
arr3
```

```
array([[[ 1,  2,  3,  4],
        [44,  5,  6,  7]],

       [[ 1,  3,  6,  7],
        [ 4,  5,  6,  7]]])
```

```
arr4 = [1,2,3,4]
```

```
arr5 = [[(1,2,3,4),(5,6,7,8)]]
```

```
arr5
```

```
[[(1, 2, 3, 4), (5, 6, 7, 8)]]
```

```
arr2
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
np.random.random((3,4)) ## outer bracker for function,inner for dimension
```

```
array([[0.18824318, 0.77064214, 0.47694799, 0.75788331],
       [0.1625749 , 0.05593507, 0.35718801, 0.0958226 ],
       [0.29187188, 0.19464167, 0.25629897, 0.79282986]])
```

```
np.full((3,3,4),10)
```

```
array([[[10, 10, 10, 10],
        [10, 10, 10, 10],
        [10, 10, 10, 10]],

       [[10, 10, 10, 10],
        [10, 10, 10, 10],
        [10, 10, 10, 10]],

       [[10, 10, 10, 10],
        [10, 10, 10, 10],
        [10, 10, 10, 10]]])
```

```
np.zeros((3,4))
```

```
array([[0., 0., 0., 0.],
       [0., 0., 0., 0.],
       [0., 0., 0., 0.]])
```

```
np.eye((4))
```

```
array([[1., 0., 0., 0.],
       [0., 1., 0., 0.],
       [0., 0., 1., 0.],
       [0., 0., 0., 1.]])
```

```
Mat1 = np.array([[1,2,3],[2,4,6]])
Mat2 = np.array([[2,4,1],[2,3,23]])

Mat3=Mat1+Mat2
print(Mat1,Mat2,Mat3)
```

```
[[1 2 3]
 [2 4 6]] [[ 2  4  1]
 [ 2  3 23]] [[ 3  6  4]
 [ 4  7 29]]
```

```
Mat3
```

```
array([[ 3,  6,  4],
       [ 4,  7, 29]])
```

```
Mat4 = np.array([[1,2],[2,3],[4,5]])
Mat4
```

```
array([[1, 2],
       [2, 3],
       [4, 5]])
```

```
Mat1*Mat4.T
```

```
array([[ 1,  4, 12],
       [ 4, 12, 30]])
```

```
np.linspace(80,90,10)
```

```
array([80.        , 81.11111111, 82.22222222, 83.33333333, 84.44444444,
       85.55555556, 86.66666667, 87.77777778, 88.88888889, 90.        ])
```

```
np.linspace(-90,90,180)
```

```
array([-90.        , -88.99441341, -87.98882682, -86.98324022,
       -85.97765363, -84.97206704, -83.96648045, -82.96089385,
       -81.95530726, -80.94972067, -79.94413408, -78.93854749,
       -77.93296089, -76.9273743 , -75.92178771, -74.91620112,
       -73.91061453, -72.90502793, -71.89944134, -70.89385475,
       -69.88826816, -68.88268156, -67.87709497, -66.87150838,
       -65.86592179, -64.8603352 , -63.8547486 , -62.84916201,
       -61.84357542, -60.83798883, -59.83240223, -58.82681564,
       -57.82122905, -56.81564246, -55.81005587, -54.80446927,
       -53.79888268, -52.79329609, -51.7877095 , -50.78212291,
       -49.77653631, -48.77094972, -47.76536313, -46.75977654,
       -45.75418994, -44.74860335, -43.74301676, -42.73743017,
       -41.73184358, -40.72625698, -39.72067039, -38.7150838 ,
       -37.70949721, -36.70391061, -35.69832402, -34.69273743,
       -33.68715084, -32.68156425, -31.67597765, -30.67039106,
       -29.66480447, -28.65921788, -27.65363128, -26.64804469,
       -25.6424581 , -24.63687151, -23.63128492, -22.62569832,
       -21.62011173, -20.61452514, -19.60893855, -18.60335196,
       -17.59776536, -16.59217877, -15.58659218, -14.58100559,
       -13.57541899, -12.5698324 , -11.56424581, -10.55865922,
        -9.55307263,  -8.54748603,  -7.54189944,  -6.53631285,
        -5.53072626,  -4.52513966,  -3.51955307,  -2.51396648,
        -1.50837989,  -0.5027933 ,   0.5027933 ,   1.50837989,
         2.51396648,   3.51955307,   4.52513966,   5.53072626,
         6.53631285,   7.54189944,   8.54748603,   9.55307263,
        10.55865922,  11.56424581,  12.5698324 ,  13.57541899,
        14.58100559,  15.58659218,  16.59217877,  17.59776536,
        18.60335196,  19.60893855,  20.61452514,  21.62011173,
        22.62569832,  23.63128492,  24.63687151,  25.6424581 ,
        26.64804469,  27.65363128,  28.65921788,  29.66480447,
        30.67039106,  31.67597765,  32.68156425,  33.68715084,
        34.69273743,  35.69832402,  36.70391061,  37.70949721,
        38.7150838 ,  39.72067039,  40.72625698,  41.73184358,
        42.73743017,  43.74301676,  44.74860335,  45.75418994,
        46.75977654,  47.76536313,  48.77094972,  49.77653631,
        50.78212291,  51.7877095 ,  52.79329609,  53.79888268,
        54.80446927,  55.81005587,  56.81564246,  57.82122905,
        58.82681564,  59.83240223,  60.83798883,  61.84357542,
        62.84916201,  63.8547486 ,  64.8603352 ,  65.86592179,
        66.87150838,  67.87709497,  68.88268156,  69.88826816,
        70.89385475,  71.89944134,  72.90502793,  73.91061453,
        74.91620112,  75.92178771,  76.9273743 ,  77.93296089,
        78.93854749,  79.94413408,  80.94972067,  81.95530726,
        82.96089385,  83.96648045,  84.97206704,  85.97765363,
        86.98324022,  87.98882682,  88.99441341,  90.        ])
```

```
x2 = np.random.randint(10,size=(3,4))
```

```
x2
```

```
array([[5, 3, 9, 8],
       [5, 9, 9, 8],
       [1, 3, 3, 5]])
```

```
x3 = np.random.randint(5,size=(3,3,4))
```

```
x3
```

```
array([[[4, 4, 2, 3],
        [0, 3, 4, 0],
        [1, 0, 2, 3]],

       [[1, 1, 3, 2],
        [1, 0, 3, 0],
        [3, 2, 3, 2]],

       [[1, 1, 3, 0],
        [2, 0, 4, 1],
        [4, 2, 3, 2]]])
```

```
import numpy as np
x= np.array([[1,2,3],[2,4,6],[7,8,9],[9,10,11]])
x
```

```
array([[ 1,  2,  3],
       [ 2,  4,  6],
       [ 7,  8,  9],
       [ 9, 10, 11]])
```

```
y= np.array([1,0,1])
```

```
y*x
```

```
array([[ 1,  0,  3],
       [ 2,  0,  6],
       [ 7,  0,  9],
       [ 9,  0, 11]])
```

```
w= np.array([1,2,3])
```

```
u=np.array([4,5])
```

```
np.reshape(w,(3,1))*u
```

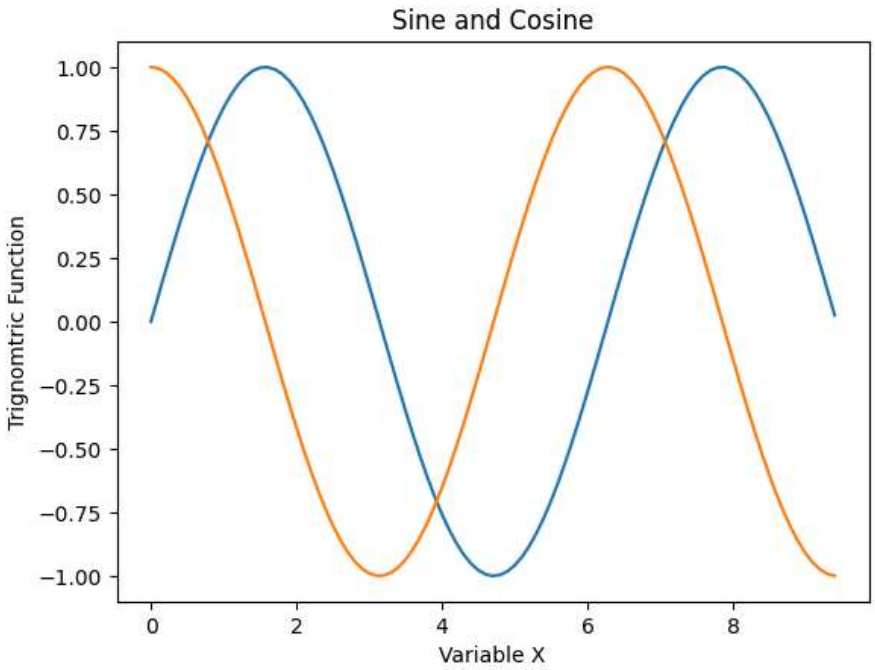
```
array([[ 4,  5],
       [ 8, 10],
       [12, 15]])
```

```
np.outer(w,u)
```

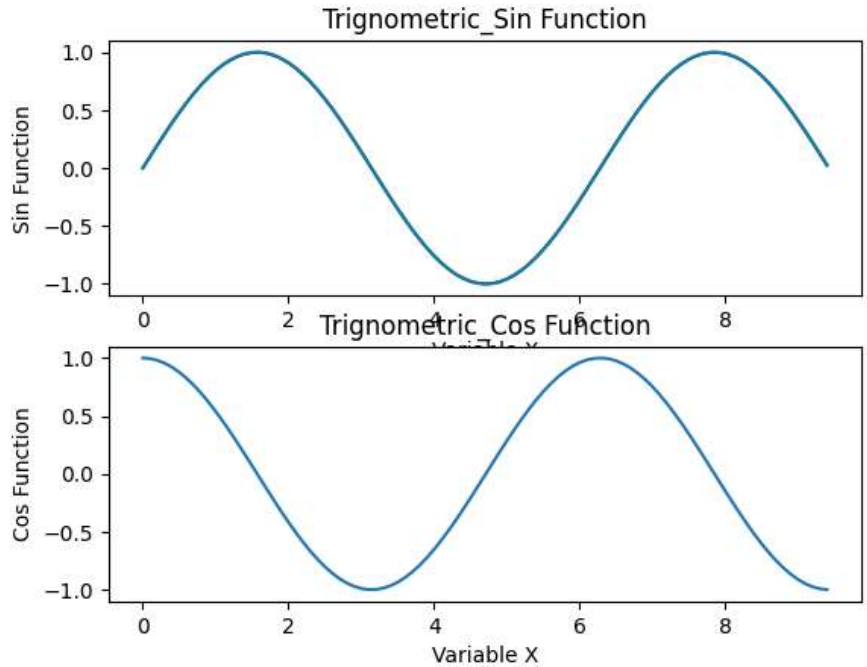
```
array([[ 4,  5],
       [ 8, 10],
       [12, 15]])
```

```
## Matplotlib.pyplot

import matplotlib.pyplot as plt

x = np.arange(0,3*np.pi,0.1)
y_sin= np.sin(x)
```

```
y_cos= np.cos(x)

plt.plot(x,y_sin)
plt.plot(x,y_cos)
plt.xlabel('Variable X')
plt.ylabel('Trignomtric Function')
plt.title('Sine and Cosine')
plt.show()
```

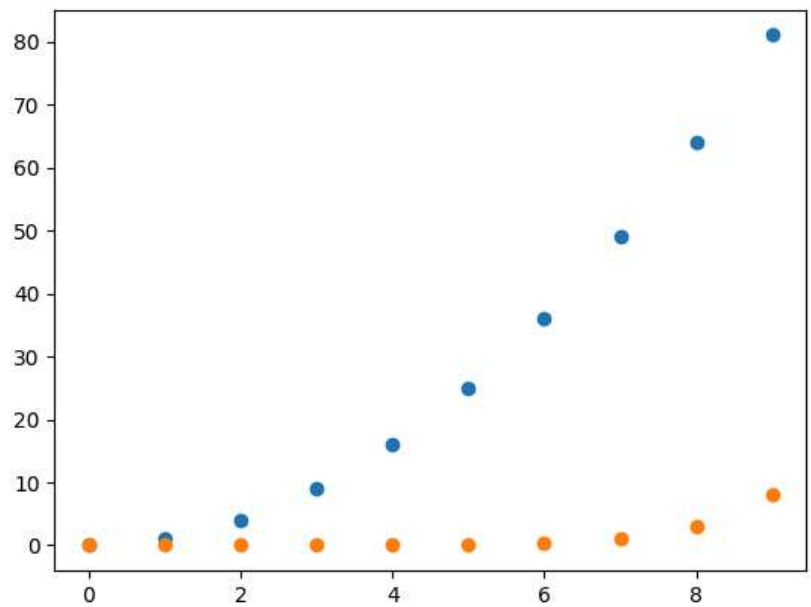



```
## Subplot
plt.subplot(2,1,1)
plt.plot(x,y_sin,color = "green")
plt.plot(x,y_sin)
plt.xlabel('Variable X')
plt.ylabel('Sin Function')
plt.title('Trignometric_Sin Function')

plt.subplot(2,1,2)
plt.plot(x,y_cos)
plt.xlabel('Variable X')
plt.ylabel('Cos Function')
plt.title('Trignometric_Cos Function')
plt.show()
```



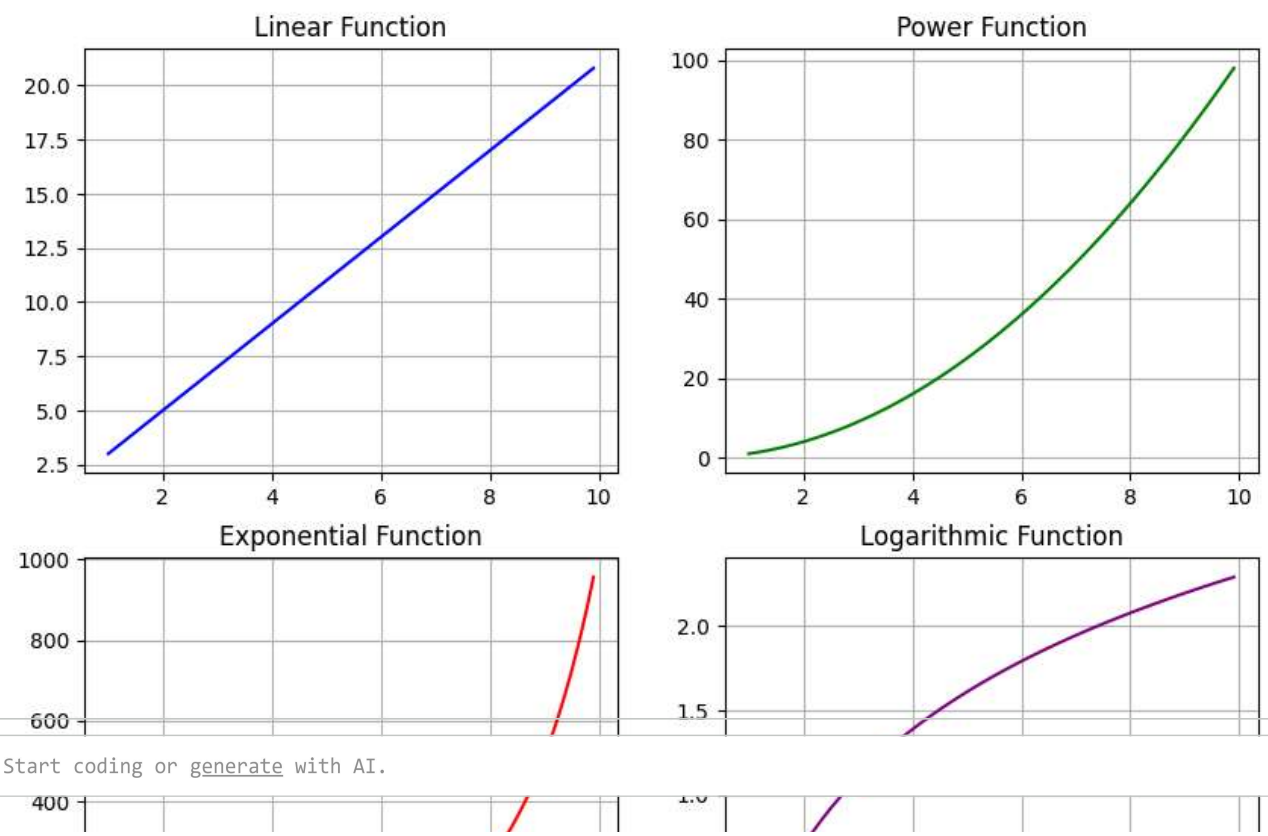

```
p = np.arange(0,10,1)
q = p**2
d = np.exp(p)

plt.scatter(p,q)
plt.scatter(p,d/1000)
```

```
<matplotlib.collections.PathCollection at 0x7cce742efbf0>
```




```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(1, 10, 0.1)

y_linear = 2*x + 1          # Linear
y_power = x**2              # Power
y_exponential = 2**x        # Exponential
y_logarithmic = np.log(x)   # Logarithmic (natural log)

plt.figure(figsize=(10, 8))
```

```
## Linear Function
plt.subplot(2, 2, 1)
plt.plot(x, y_linear, color='blue')
plt.title("Linear Function")
plt.grid(True)

# Power Function
plt.subplot(2, 2, 2)
plt.plot(x, y_power, color='green')
plt.title("Power Function")
plt.grid(True)

#Exponential Function
plt.subplot(2, 2, 3)
plt.plot(x, y_exponential, color='red')
plt.title("Exponential Function")
plt.grid(True)

#Logarithmic Function
plt.subplot(2, 2, 4)
plt.plot(x, y_logarithmic, color='purple')
plt.title("Logarithmic Function")
plt.grid(True)

plt.show()
```



Start coding or generate with AI.